



筑波大学

University of Tsukuba

Data Engineering I

Quiz 6

Mamanchuk Mykola, SID.202420671

November 18, 2024

Task 1: DBSCAN Clustering

Problem Statement

Given the following distance matrix measured by the Euclidean distance:

Table 1: Distance Matrix

	x_1	x_2	x_3	x_4	x_5	x_6	x_7
x_1	0.00	7.59	8.11	3.01	1.00	9.02	1.00
x_2	7.59	0.00	1.10	6.33	7.50	1.20	8.22
x_3	8.11	1.10	0.00	6.33	8.34	1.51	7.08
x_4	3.01	6.33	6.33	0.00	3.00	9.63	3.00
x_5	1.00	7.50	8.34	3.00	0.00	8.42	1.00
x_6	9.02	1.20	1.51	9.63	8.42	0.00	8.33
x_7	1.00	8.22	7.08	3.00	1.00	8.33	0.00

We apply the DBSCAN algorithm with parameters $\varepsilon = 1.2$ and $\text{MinPts} = 3$.

Questions and Answers

(i) **Find all core points.**

Solution: For each point, we count the number of points (including itself) within $\varepsilon = 1.2$ distance.

- **Point x_1 :** Neighbors within ε : x_1, x_5, x_7 (distance ≤ 1.0). Count = 3. Therefore, x_1 is a core point.
- **Point x_2 :** Neighbors within ε : x_2, x_3 (1.10), x_6 (1.20). Count = 3. Therefore, x_2 is a core point.
- **Point x_5 :** Neighbors within ε : x_5, x_1, x_7 (1.00). Count = 3. Therefore, x_5 is a core point.

- **Point x_7 :** Neighbors within ε : x_7, x_1, x_5 (1.00). Count = 3. Therefore, x_7 is a core point.
- **Points x_3, x_4, x_6 :** Each has fewer than 3 neighbors within ε . Therefore, they are not core points.

Answer: The core points are x_1, x_2, x_5 , and x_7 .

(ii) **Find all clusters and noise points.**

Solution: We start clustering from the core points.

- **Cluster 1:** Starting from core point x_1 , its neighbors within ε are x_1, x_5, x_7 . These points form Cluster 1.
- **Cluster 2:** Starting from core point x_2 , its neighbors within ε are x_2, x_3, x_6 . Points x_3 and x_6 are reachable from x_2 and are added to Cluster 2. Note that x_3 and x_6 are border points.
- **Noise Point:** Point x_4 does not belong to any cluster and is not a core point. Therefore, x_4 is a noise point.

Answer

The DBSCAN algorithm identified two clusters and one noise point based on the given distance matrix and parameters.

Core Points: x_1, x_2, x_5, x_7

Clusters:

- Cluster 1: x_1, x_5, x_7
- Cluster 2: x_2, x_3, x_6

Noise Point: x_4

Task 2: Prototype-Based Cohesion, Separation, and NMI

Problem Statement

Given the following 2-dimensional data points:

Table 2: Data Points

Point	x	y
x_1	35	90
x_2	78	100
x_3	80	80
x_4	79	78
x_5	91	80
x_6	20	80
x_7	52	60
x_8	105	50

We have the following clusters:

- **Cluster 1:** $\{x_1, x_6, x_7\}$
- **Cluster 2:** $\{x_2, x_3, x_4, x_5, x_8\}$

And the following class labels:

- **Label 1:** $\{x_1, x_2, x_6\}$
- **Label 2:** $\{x_3, x_4, x_5, x_7, x_8\}$

Questions and Answers

(i) **Compute the prototype-based cohesion of the clustering result.**

Solution: Cohesion is calculated as the sum of squared Euclidean distances between each point and its cluster centroid.

- **Cluster 1 Centroid:**

$$\begin{aligned}\text{Centroid}_1 &= \left(\frac{35 + 20 + 52}{3}, \frac{90 + 80 + 60}{3} \right) \\ &= (35.67, 76.67)\end{aligned}$$

- **Cluster 1 Cohesion:**

$$\text{Cohesion}_1 = \sum_{i \in \text{Cluster 1}} \|x_i - \text{Centroid}_1\|^2 = 980.33$$

- **Cluster 2 Centroid:**

$$\begin{aligned}\text{Centroid}_2 &= \left(\frac{78 + 80 + 79 + 91 + 105}{5}, \frac{100 + 80 + 78 + 80 + 50}{5} \right) \\ &= (86.6, 77.6)\end{aligned}$$

- **Cluster 2 Cohesion:**

$$\text{Cohesion}_2 = \sum_{i \in \text{Cluster 2}} \|x_i - \text{Centroid}_2\|^2 = 1807.40$$

- **Total Cohesion:**

$$\text{Total Cohesion} = \text{Cohesion}_1 + \text{Cohesion}_2 = 980.33 + 1807.40 = 2787.73$$

(ii) **Compute the prototype-based separation of the clusters.**

Solution: Separation is calculated as the squared Euclidean distance between the centroids of the clusters.

$$\begin{aligned}\text{Separation} &= \|\text{Centroid}_1 - \text{Centroid}_2\|^2 \\ &= (86.6 - 35.67)^2 + (77.6 - 76.67)^2 \\ &= 2595.08\end{aligned}$$

Answer: The prototype-based separation is **2595.08**.

(iii) **Compute the Normalized Mutual Information (NMI) of the clusters.**

Solution: NMI measures the similarity between the clustering and the true labels.

- **Construct the Contingency Table:**

Table 3: Contingency Table

	Cluster 1	Cluster 2	Total
Label 1	2	1	3
Label 2	1	4	5
Total	3	5	8

- Compute Mutual Information (MI):

$$\begin{aligned}
 MI &= \sum_{i=1}^2 \sum_{j=1}^2 \frac{n_{ij}}{N} \log_2 \left(\frac{N \cdot n_{ij}}{n_i \cdot n_j} \right) \\
 &= \frac{2}{8} \log_2 \left(\frac{8 \cdot 2}{3 \cdot 3} \right) + \frac{1}{8} \log_2 \left(\frac{8 \cdot 1}{3 \cdot 5} \right) \\
 &\quad + \frac{1}{8} \log_2 \left(\frac{8 \cdot 1}{5 \cdot 3} \right) + \frac{4}{8} \log_2 \left(\frac{8 \cdot 4}{5 \cdot 5} \right) \\
 &= 0.159
 \end{aligned}$$

- Compute Entropies $H(L)$ and $H(C)$:

$$\begin{aligned}
 H(L) &= - \left(\frac{3}{8} \log_2 \frac{3}{8} + \frac{5}{8} \log_2 \frac{5}{8} \right) = 0.954 \\
 H(C) &= - \left(\frac{3}{8} \log_2 \frac{3}{8} + \frac{5}{8} \log_2 \frac{5}{8} \right) = 0.954
 \end{aligned}$$

- Compute NMI:

$$NMI = \frac{2 \times MI}{H(L) + H(C)}$$

Answer: The Normalized Mutual Information (NMI) is **0.166**.

Listing assignment.py

```

1  import numpy as np
2
3  #####
4  ### ASSIGNMENT 1 ###
5  #####
6
7  # Distance matrix
8  distance_matrix = np.array([
9      [0.00, 7.59, 8.11, 3.01, 1.00, 9.02, 1.00],
10     [7.59, 0.00, 1.10, 6.33, 7.50, 1.20, 8.22],
11     [8.11, 1.10, 0.00, 6.33, 8.34, 1.51, 7.08],
12     [3.01, 6.33, 6.33, 0.00, 3.00, 9.63, 3.00],
13     [1.00, 7.50, 8.34, 3.00, 0.00, 8.42, 1.00],
14     [9.02, 1.20, 1.51, 9.63, 8.42, 0.00, 8.33],
15     [1.00, 8.22, 7.08, 3.00, 1.00, 8.33, 0.00],
16 ])
17
18 Eps = 1.2
19 MinPts = 3
20 n_points = distance_matrix.shape[0]
21
22 # Initialize labels (-1 means unvisited)
23 labels = np.full(n_points, -1)
24
25 # List to keep track of core points
26 core_points = []
27
28 # Step 1: Find core points
29 for i in range(n_points):
30     neighbors = np.where(distance_matrix[i] <= Eps)[0]
31     if len(neighbors) >= MinPts:
32         core_points.append(i)
33
34 # Step 2: Cluster assignment
35 cluster_id = 0
36 for core_point in core_points:
37     if labels[core_point] == -1:
38         # Start a new cluster
39         labels[core_point] = cluster_id
40         # Initialize the list of points to visit

```

```

41     points_to_visit = [core_point]
42     while points_to_visit:
43         current_point = points_to_visit.pop()
44         neighbors = np.where(distance_matrix[current_point] <= Eps)[0]
45         for neighbor in neighbors:
46             if labels[neighbor] == -1:
47                 labels[neighbor] = cluster_id
48                 if neighbor in core_points:
49                     points_to_visit.append(neighbor)
50     cluster_id += 1
51
52 # Map indices to point names
53 point_names = ['x1', 'x2', 'x3', 'x4', 'x5', 'x6', 'x7']
54 clusters = {}
55 noise = []
56 for idx, label in enumerate(labels):
57     if label != -1:
58         cluster = clusters.get(label, [])
59         cluster.append(point_names[idx])
60         clusters[label] = cluster
61     else:
62         noise.append(point_names[idx])
63
64 # Print results
65 print("Core Points:", [point_names[i] for i in core_points])
66 print("\nClusters:")
67 for cluster_id, members in clusters.items():
68     print(f"Cluster {cluster_id + 1}: {' ', ' '.join(members)}")
69 print("\nNoise Points:", ' ', ' '.join(noise))
70
71
72 #####
73 ### ASSIGNMENT 2 ###
74 #####
75
76 print("\n")
77
78 # Data points
79 data = {
80     'x1': np.array([35, 90]),
81     'x2': np.array([78, 100]),
82     'x3': np.array([80, 80]),
83     'x4': np.array([79, 78]),
84     'x5': np.array([91, 80]),
85     'x6': np.array([20, 80]),
86     'x7': np.array([52, 60]),
87     'x8': np.array([105, 50])
88 }
89
90 # Clusters
91 cluster1 = ['x1', 'x6', 'x7']
92 cluster2 = ['x2', 'x3', 'x4', 'x5', 'x8']
93
94 ### Prototype-Based Cohesion
95
96 # Function to compute centroid
97 def compute_centroid(cluster_points):
98     return np.mean(cluster_points, axis=0)
99
100 # Function to compute cohesion
101 def compute_cohesion(cluster_points, centroid):
102     distances = np.linalg.norm(cluster_points - centroid, axis=1)
103     squared_distances = distances ** 2
104     return np.sum(squared_distances)
105
106 # Cluster 1
107 cluster1_points = np.array([data[pt] for pt in cluster1])
108 centroid1 = compute_centroid(cluster1_points)
109 cohesion1 = compute_cohesion(cluster1_points, centroid1)
110
111 # Cluster 2
112 cluster2_points = np.array([data[pt] for pt in cluster2])
113 centroid2 = compute_centroid(cluster2_points)

```

```

114 cohesion2 = compute_cohesion(cluster2_points, centroid2)
115
116 # Total Cohesion
117 total_cohesion = cohesion1 + cohesion2
118
119 # Round to second decimal place
120 total_cohesion = round(total_cohesion, 2)
121
122 print(f"Prototype-Based Cohesion: {total_cohesion}")
123
124 ### Prototype-Based Separation
125
126 # Compute squared Euclidean distance between centroids
127 centroid_distance = np.linalg.norm(centroid1 - centroid2)
128 separation = centroid_distance ** 2
129
130 # Round to second decimal place
131 separation = round(separation, 2)
132
133 print(f"Prototype-Based Separation: {separation}")
134
135 ### Normalized Mutual Information (NMI)
136
137 # Data for contingency table, n_ij counts
138 n11 = 2 # Label 1, Cluster 1
139 n12 = 1 # Label 1, Cluster 2
140 n21 = 1 # Label 2, Cluster 1
141 n22 = 4 # Label 2, Cluster 2
142
143 n1_dot = n11 + n12 # Total for Label 1
144 n2_dot = n21 + n22 # Total for Label 2
145 n_dot1 = n11 + n21 # Total for Cluster 1
146 n_dot2 = n12 + n22 # Total for Cluster 2
147 N = n1_dot + n2_dot # Total number of points
148
149 # Calculate MI
150 import math
151
152 def log2(x):
153     return math.log(x) / math.log(2)
154
155 def mutual_information():
156     MI = 0
157     # Terms where n_ij > 0
158     n_ij_list = [(n11, n1_dot, n_dot1), (n12, n1_dot, n_dot2),
159                 (n21, n2_dot, n_dot1), (n22, n2_dot, n_dot2)]
160     for n_ij, n_i_dot, n_dot_j in n_ij_list:
161         if n_ij > 0:
162             MI += (n_ij / N) * log2((N * n_ij) / (n_i_dot * n_dot_j))
163     return MI
164
165 MI = mutual_information()
166
167 # Calculate entropies
168 def entropy(n_i_dot):
169     p = n_i_dot / N
170     return -p * log2(p)
171
172 H_L = entropy(n1_dot) + entropy(n2_dot)
173 H_C = entropy(n_dot1) + entropy(n_dot2)
174
175 # Calculate NMI
176 NMI = (2 * MI) / (H_L + H_C)
177
178 # Round to third decimal place
179 NMI = round(NMI, 3)
180
181 print(f"NMI: {NMI}")

```